

Banco de Preguntas para la Defensa

Preguntas típicas del jurado, con respuestas modelo

Consejo: no memorices las respuestas palabra por palabra. Léelas, entiéndelas, y practica decirlas con TUS palabras frente a un espejo o a un amigo. El jurado nota la diferencia entre entender y recitar.

A. Sobre las tecnologías elegidas

1. ¿Por qué PHP y no un framework como Laravel, o Node.js/React?

Porque el alcance del proyecto no lo requería y priorizamos tres cosas: facilidad de despliegue (PHP corre en cualquier hosting económico o en XAMPP, importante para una PyME venezolana), curva de aprendizaje, y control total del código sin depender de librerías externas. Un framework agrega valor en equipos grandes y proyectos que crecen mucho; aquí habría agregado complejidad sin beneficio proporcional. La arquitectura sí toma ideas de los frameworks: separación de vistas y endpoints JSON, layout compartido y migraciones automáticas de base de datos.

2. ¿Por qué MySQL?

Es una base de datos relacional: el dominio del proyecto (facturas con renglones, pagos, clientes, inventario) es naturalmente relacional y necesita integridad referencial y transacciones ACID. Además es gratuita, madura y la más común en hostings PHP.

3. ¿Por qué JavaScript "puro" y no React/Vue?

Por el mismo criterio: el proyecto necesita interactividad concreta (carrito, buscador, cálculos reactivos) que se resuelve bien con JavaScript nativo y `fetch()`. Se organizó en módulos por página con funciones reutilizables en `common.js` para no duplicar código.

B. Sobre la tasa BCV (casi seguro la preguntan)

4. ¿De dónde sale la tasa? ¿Qué pasa si no hay internet?

La tasa se consulta una vez al día a una API pública que replica la tasa oficial del BCV, y se guarda en la tabla `tasas_bcv` (caché diario). El sistema tiene una cadena de respaldo de tres niveles: (1) si el administrador fijó una tasa manual hoy, esa manda; (2) si no, se usa la de la API; (3) si la API falla o no hay internet, se usa la última tasa conocida. El sistema nunca se detiene por falta de tasa.

5. Si la tasa cambia mañana, ¿los reportes viejos cambian?

No. Cada movimiento y cada factura guardan la tasa con la que se hicieron (`tasa_bcv`) y el monto en bolívares ya calculado (`monto_bs`). El histórico es inmutable, como exige cualquier práctica contable.

6. ¿La conversión la hace el navegador o el servidor?

El navegador la muestra en tiempo real para la experiencia de usuario, pero el valor que se guarda lo calcula **el servidor**. Nunca se confía en datos calculados por el cliente, porque podrían manipularse.

C. Sobre la facturación (POS)

7. ¿Cómo garantizan que no quede una factura a medias si se va la luz?

Con transacciones SQL. Todo el guardado (cabecera, ítems, pagos, movimientos y descuento de stock) ocurre entre `beginTransaction()` y `commit()`. Si cualquier paso falla, se ejecuta `rollback()` y la base de datos queda exactamente como estaba. Es la propiedad de atomicidad de ACID.

8. ¿Cómo evitan dos facturas con el mismo número?

Con dos mecanismos: el correlativo automático se calcula con `SELECT ... FOR UPDATE` dentro de la transacción (bloquea a otro cajero simultáneo), y además la columna `numero_factura` tiene un índice UNIQUE en la base de datos, que es la garantía final.

9. ¿Cómo funcionan los pagos mixtos?

Una factura puede tener varias líneas de pago, cada una con su método (efectivo, pago móvil, transferencia...), su moneda (USD o Bs) y su monto. El sistema convierte los pagos en Bs a su equivalente en dólares con la tasa del día y valida que la suma cubra el total con una tolerancia de \$0.10 por redondeos. Si no cuadra, no deja guardar.

10. ¿Qué pasa si venden algo sin stock?

Decisión de negocio tomada con el usuario: el sistema **avisa** que el stock quedará negativo pero **permite** la venta, porque en la operación real el inventario físico no siempre está actualizado y no se debe frenar una venta por un error administrativo. El stock negativo queda visible en rojo en el catálogo para forzar la corrección. (Un control estricto sería trivial de activar: es un `if` en `guardar_factura.php`.)

D. Sobre seguridad

11. ¿Cómo protegen las contraseñas?

Con `password_hash()` de PHP, que usa BCRYPT: un algoritmo de hash con "sal" aleatoria y costo computacional configurable. Ni siquiera el administrador de la base de datos puede ver las contraseñas.

12. ¿Cómo previenen inyección SQL?

Con **prepared statements** de PDO: la consulta y los datos viajan por separado al motor, de modo que un dato malicioso (`' OR 1=1 --`) nunca se interpreta como SQL. Todos los endpoints usan este patrón.

13. ¿Puede alguien usar el sistema sin loguearse?

No. Las páginas incluyen `auth.php` (redirige al login) y los endpoints JSON incluyen `verificar_sesion_api.php` (devuelven error). Ambos verifican la sesión del lado del servidor.

E. Sobre la base de datos

14. ¿Qué normalización tiene el modelo?

Está en tercera forma normal (3FN) en lo esencial: catálogos separados (conceptos, clientes, proveedores, bancos) referenciados por llaves foráneas, sin datos repetidos. Hay una desnormalización **intencional**: `factura_items` copia la descripción y el precio del momento de la venta, porque una factura emitida es un documento inmutable aunque el catálogo cambie después.

15. ¿Qué pasa si borran un cliente que tiene facturas?

La llave foránea está configurada `ON DELETE SET NULL` : el historial contable se conserva y la factura queda como "consumidor final". Los conceptos con historial, en cambio, tienen `RESTRICT` : no se pueden borrar.

16. ¿Cómo se instala el sistema en una máquina nueva?

Se copia el código y listo: en cada conexión, `instalador.php` verifica contra `INFORMATION_SCHEMA` que existan la base de datos, las tablas, las columnas y las llaves foráneas, y crea o repara lo que falte. Es el equivalente a las migraciones automáticas de los frameworks.

F. Sobre metodología y gestión

17. ¿Qué metodología usaron?

Desarrollo iterativo incremental inspirado en Scrum: el requerimiento se organizó en un backlog de 4 épicas (arquitectura base, transacciones simples, facturación POS, consultas y dashboards) descompuestas en tareas. Antes de programar se resolvieron las preguntas de definición con el usuario (fuente de la tasa, política de inventario, formato de impresión, pagos mixtos), y cada módulo se probó al completarse.

18. ¿Cómo probaron el sistema?

Pruebas funcionales de caja negra sobre los endpoints (casos válidos e inválidos: pagos que no cuadran, stock insuficiente, correlativos duplicados, sesión expirada) y pruebas de integración del flujo completo: vender en el POS y verificar que el stock baje, el movimiento aparezca en los listados y los totales cuadren en el resumen y las estadísticas.

G. Trabajo futuro (si preguntan "¿qué le falta?")

Responder con seguridad — tener trabajo futuro identificado es señal de madurez, no de debilidad:

1. **Roles de usuario** (administrador vs cajero) con permisos diferenciados.
2. **Compras que alimenten el inventario**: que un egreso de compra de mercancía aumente el stock automáticamente.
3. **Reportes exportables** (PDF/Excel) y cierre de caja diario.
4. **Más KPIs**: producto más vendido, utilidad neta por producto, ticket promedio.
5. **Copias de seguridad automáticas** de la base de datos.

Los 5 conceptos que DEBES dominar sí o sí

1. **Flujo petición → endpoint → JSON → tabla** (sección 3 de la guía de arquitectura).
2. **Transacciones ACID** en `guardar_factura.php` (pregunta 7).
3. **Cadena de respaldo de la tasa BCV** (pregunta 4) y por qué se guarda por transacción (pregunta 5).
4. **Prepared statements** y BCRYPT (preguntas 11-12).
5. **El diagrama ER**: poder dibujarlo de memoria, al menos facturas ↔ items ↔ pagos y movimientos ↔ conceptos.